INVENTORY MANAGEMENT SYSTEM FOR EMART GROCERY SHOP

A MINI PROJECT REPORT

Submitted by

EDWIN MARIO. A
311124205015

BACHELOR OF TECHNOLOGY
IN
INFORMATION TECHNOLOGY

LOYOLA-ICAM COLLEGE OF ENGINEERING AND TECHNOLOGY

ANNA UNIVERSITY: CHENNAI 600 025

MARCH 2026

BONAFIDE CERTIFICATE

Certified that the Mini project report on **"INVENTORY MANAGEMENT SYSTEM FOR EMART GROCERY SHOP"** is the bona fide work of **EDWIN MARIO. A 311124205015,** who carried out the project work under my supervision.

SIGNATURE

Dr G Shobana
Project Coordinator
Department of Information Technology
Loyola-ICAM College of Engineering
and Technology
Loyola Campus,
Nungambakkam, Chennai-34

SIGNATURE

Ms. Sherril Sophie Maria Vincent
Head of the Department
Department of Information Technology
Loyola-ICAM College of Engineering
and Technology
Loyola Campus,
Nungambakkam, Chennai-34

ACKNOWLEDGEMENT

First of all, we thank the Lord Almighty for showering His blessings on us, which enabled us to complete this project successfully.

We express our sincere gratitude to our beloved and respected **Director, Rev. Dr. Justine SJ.** We wish to acknowledge with thanks to our college, **Dean of Students, Rev. Dr. Albert William SJ**, for his support and encouragement.

We extend our sincere gratitude to **Dr. A. Baskar, Principal**, for his motivation and support towards this project.

We express our sincere gratitude to the **Head of the Department** of Information Technology, **Ms. Sherril Sophie Maria Vincent**, for her constant support, motivation, encouragement and being a source of inspiration throughout this project and during the course of this semester.

Our sincere thanks to our **Project Coordinator Dr. G Shobana** for their valuable guidance in shaping this project.

We also take this opportunity to thank all the **Faculty and Non-teaching staff members** of Department of Information Technology for their constant support. Finally, we thank each and every one who helped us to complete this project.

EDWIN MARIO. A

ABSTRACT

The Inventory Management System for EMart Grocery Shop is a GUI-based desktop application developed using Python (Tkinter) as the front-end and MySQL as the back-end relational database. The system is designed to digitize and automate the core operations of a retail grocery store, including product catalogue management, supplier tracking, real-time stock monitoring, customer billing, and automated end-of-day (EOD) batch processing for stock replenishment.

The database design follows normalization principles up to Third Normal Form (3NF) to eliminate redundancy and ensure data integrity across all tables. The application implements enterprise-grade database features including views for daily sales and low-stock monitoring, an audit trigger that logs every stock modification to a dedicated audit log table, and reusable SQL functions for computing stock valuations.

Complex business workflows are handled through PL/SQL Stored Procedures, including a complete billing procedure (sp_GenerateBill) that wraps all transactional operations and an EOD batch procedure (sp_EOD_Reorder) that automatically flags products below reorder level for procurement. The system explicitly demonstrates the four ACID properties — Atomicity, Consistency, Isolation, and Durability — through annotated SQL transaction scripts with rollback handling.

The application provides a user-friendly Tkinter GUI with dedicated modules for product management, supplier management, stock control, billing, and report generation. This project serves as a real-world demonstration of database application development, integrating frontend design, SQL engineering, and enterprise database best practices.

Keywords: *Inventory Management, MySQL, Python Tkinter, PL/SQL, Normalization, ACID Properties, Triggers, Stored Procedures, GUI Application.*

TABLE OF CONTENTS

TITLE	PAGE NO.
ABSTRACT	i
LIST OF FIGURES	ii
LIST OF TABLES	iii
LIST OF ABBREVIATIONS	iv
CHAPTER 1 – INTRODUCTION	1
1.1 Problem Statement	1
1.2 Objectives	2
CHAPTER 2 – REQUIREMENTS	3
2.1 Functional Requirements	3
2.2 Non-Functional Requirements	4
CHAPTER 3 – ANALYSIS	5
3.1 Bull Diagram	5
3.2 Octopus Diagram	6
CHAPTER 4 – SYSTEM DESIGN	7
4.1 Entity-Relationship (ER) Diagram	7
4.2 System Architecture Diagram	9
4.3 Data Flow Diagram	10
CHAPTER 5 – DATABASE DESIGN & NORMALIZATION	12
5.1 Normalization (1NF to 3NF)	12
5.2 Complete Table Schemas	14
CHAPTER 6 – VIEWS, TRIGGERS & FUNCTIONS	18
6.1 Views	18
6.2 Triggers (Audit)	19
6.3 Functions	20
CHAPTER 7 – STORED PROCEDURES & EOD BATCH	21
7.1 sp_GenerateBill	21
7.2 sp_EOD_Reorder	23
CHAPTER 8 – ACID PROPERTIES DEMONSTRATION	24
CHAPTER 9 – MODULE DESCRIPTION	27
CHAPTER 10 – CONCLUSION	30
CHAPTER 11 – REFERENCES	31
CHAPTER 12 – APPENDICES	32

LIST OF FIGURES

FIGURE NO.	NAME	PAGE NO.
3.1	Bull Diagram – EMart Inventory System	5
3.2	Octopus Diagram – Functional Overview	6
4.1	Entity-Relationship (ER) Diagram	7
4.2	System Architecture Diagram	9
4.3	Data Flow Diagram – Level 0	10
4.4	Data Flow Diagram – Level 1	11
9.1	Dashboard – Home Screen	27
9.2	Product Management Module	28
9.3	Stock Management Module	28
9.4	Billing Module	29
9.5	Reports Module – Low Stock Alert View	29

LIST OF TABLES

TABLE NO.	NAME	PAGE NO.
2.1	Functional Requirements	3
5.1	Unnormalized Product-Supplier Data (UNF)	12
5.2	After 1NF – Atomic Values	13
5.3	After 2NF – Partial Dependency Removed	13
5.4	After 3NF – Transitive Dependency Removed	14
8.1	ACID Properties Summary	26

LIST OF ABBREVIATIONS

ABBREVIATION	FULL FORM
GUI	Graphical User Interface
SQL	Structured Query Language
PL/SQL	Procedural Language / Structured Query Language
ACID	Atomicity, Consistency, Isolation, Durability
ER	Entity Relationship
DFD	Data Flow Diagram
UML	Unified Modelling Language
EOD	End of Day
CRUD	Create, Read, Update, Delete
3NF	Third Normal Form
IDE	Integrated Development Environment
DBMS	Database Management System
UNF	Unnormalized Form
1NF	First Normal Form
2NF	Second Normal Form

CHAPTER 1: INTRODUCTION

1.1 Problem Statement

Retail grocery shops like EMart face significant operational challenges when relying on manual or paper-based inventory tracking. Without a systematic approach, staff are unable to accurately determine current stock levels, leading to frequent out-of-stock situations on fast-moving items and wasteful overstocking of slow-moving products. Billing errors are common when prices are updated manually, and there is no automated mechanism to alert management when products approach reorder levels.

Additionally, the absence of a structured supplier management system makes it difficult to track which supplier provides which product, at what price, and with what lead time. Manual billing processes are slow, error-prone, and provide no transaction history for audit or analysis. There is no centralized data store to generate meaningful business reports such as daily sales summaries, top-selling products, or revenue trends.

This project addresses these problems by developing a GUI-based Inventory Management System for EMart Grocery Shop that integrates a relational MySQL database with a Python Tkinter desktop application. The system provides real-time stock tracking, automated billing, supplier management, comprehensive reporting, and enterprise-grade database features including audit triggers, stored procedures, and ACID-compliant transactions.

1.2 Objectives

- Design and implement a normalized relational database (up to 3NF) for grocery inventory management.
- Develop an intuitive Python Tkinter GUI for all CRUD operations across all modules.
- Implement database views for reporting — low stock alerts, daily sales, and product performance.
- Create audit triggers that automatically log every stock change to a dedicated audit table.
- Develop SQL functions that compute stock valuations and apply loyalty discounts.
- Build PL/SQL stored procedures for complex business logic including billing and EOD batch reorder processing.
- Demonstrate all four ACID properties using annotated transaction scripts with rollback handling.
- Provide a scalable, modular codebase extensible with new features in future iterations.

CHAPTER 2: REQUIREMENTS

2.1 Functional Requirements

Table 2.1 below lists all functional requirements of the EMart Inventory Management System.

REQ. ID	REQUIREMENT	DESCRIPTION
FR-01	Product Management	Add, edit, delete, and search products with category assignment
FR-02	Category Management	Create and manage product categories
FR-03	Supplier Management	Maintain supplier records and their product mappings
FR-04	Stock Tracking	Real-time stock quantity view with last-updated timestamp
FR-05	Low Stock Alerts	Automatically flag products below reorder level via database view
FR-06	Customer Billing	Generate itemized bills with automatic stock deduction
FR-07	Customer Management	Register customers and track loyalty points
FR-08	EOD Batch Processing	Automated stored procedure to flag reorder requests at day end
FR-09	Audit Logging	Trigger-based automatic logging of all stock changes
FR-10	Sales Reports	View daily revenue, total bills, and product-wise sales
FR-11	Stock Valuation	SQL function to compute total stock value per product
FR-12	ACID Transactions	All billing operations wrapped in atomic DB transactions

Table 2.1: Functional Requirements

2.2 Non-Functional Requirements

- Performance: The system should load any GUI module within 2 seconds and execute SQL queries within 500 milliseconds on standard hardware.
- Reliability: All database transactions must be ACID-compliant to prevent data corruption or partial updates.
- Usability: The GUI must be intuitive and navigable by non-technical grocery staff with minimal training.
- Scalability: The database schema must support up to 10,000 product SKUs and 1 million billing records without redesign.
- Security: Database credentials must be stored in a configuration file and not hard-coded in the application source.
- Maintainability: Code must follow modular architecture with separate files per module for easy maintenance.
- Data Integrity: Foreign key constraints, NOT NULL constraints, and CHECK constraints must enforce data consistency at the database level.

CHAPTER 3: ANALYSIS

3.1 Bull Diagram

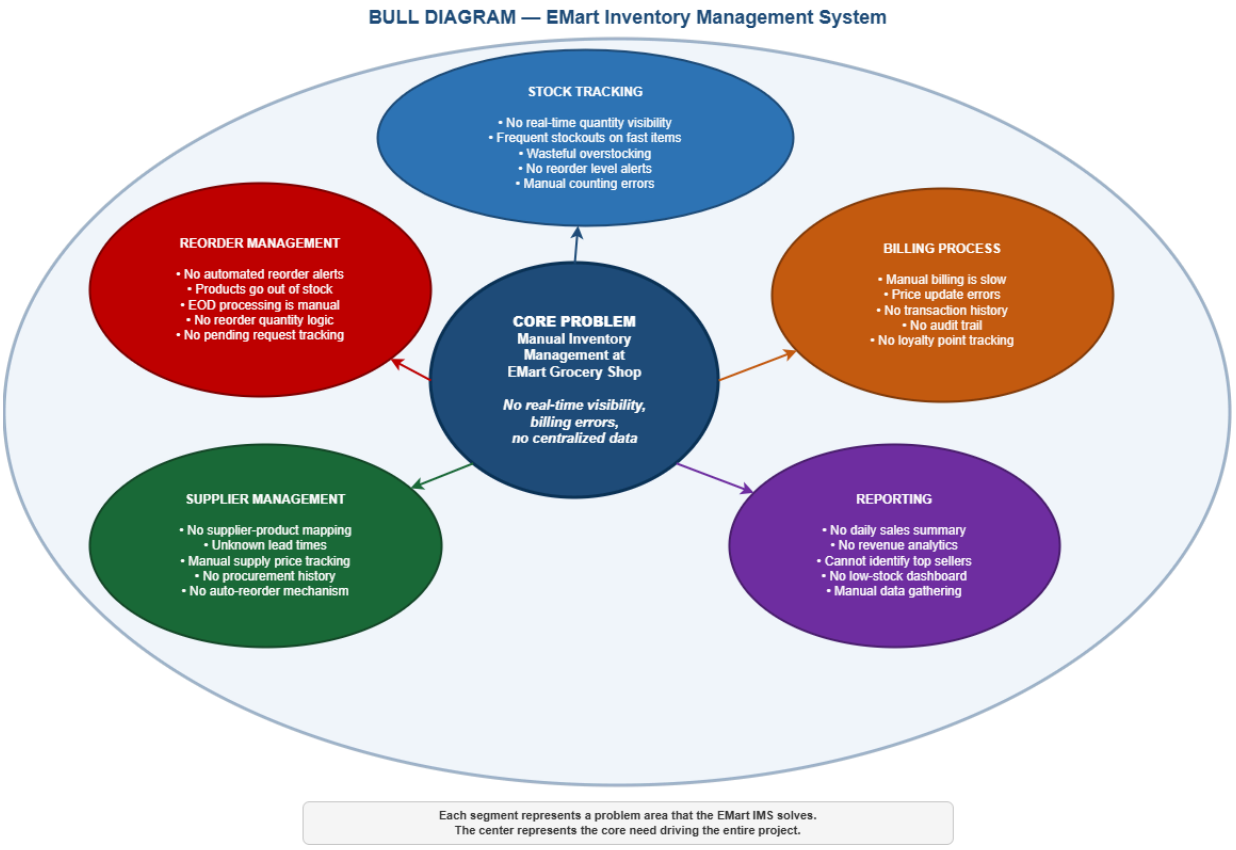
The Bull Diagram (also called a Need Analysis diagram) represents the central problem of the project surrounded by its context areas. It visually depicts the core need at the center — 'Inventory Management for EMart' — surrounded by the key operational problem areas that the system must address.

Core Problem (Center of Bull Diagram):

Manual Inventory Management at EMart Grocery Shop — leading to stockouts, billing errors, and lack of operational visibility.

Surrounding Problem Areas:

- Stock Tracking: No real-time visibility into product quantities leading to stockouts and overstock situations.
- Billing Process: Manual billing is slow, error-prone, and provides no transaction audit trail.
- Supplier Management: No centralized record of supplier-product mappings and supply pricing.
- Reporting: Management cannot access daily sales, revenue, or stock status without manual counting.
- Reorder Management: No automated mechanism to flag products that need restocking.



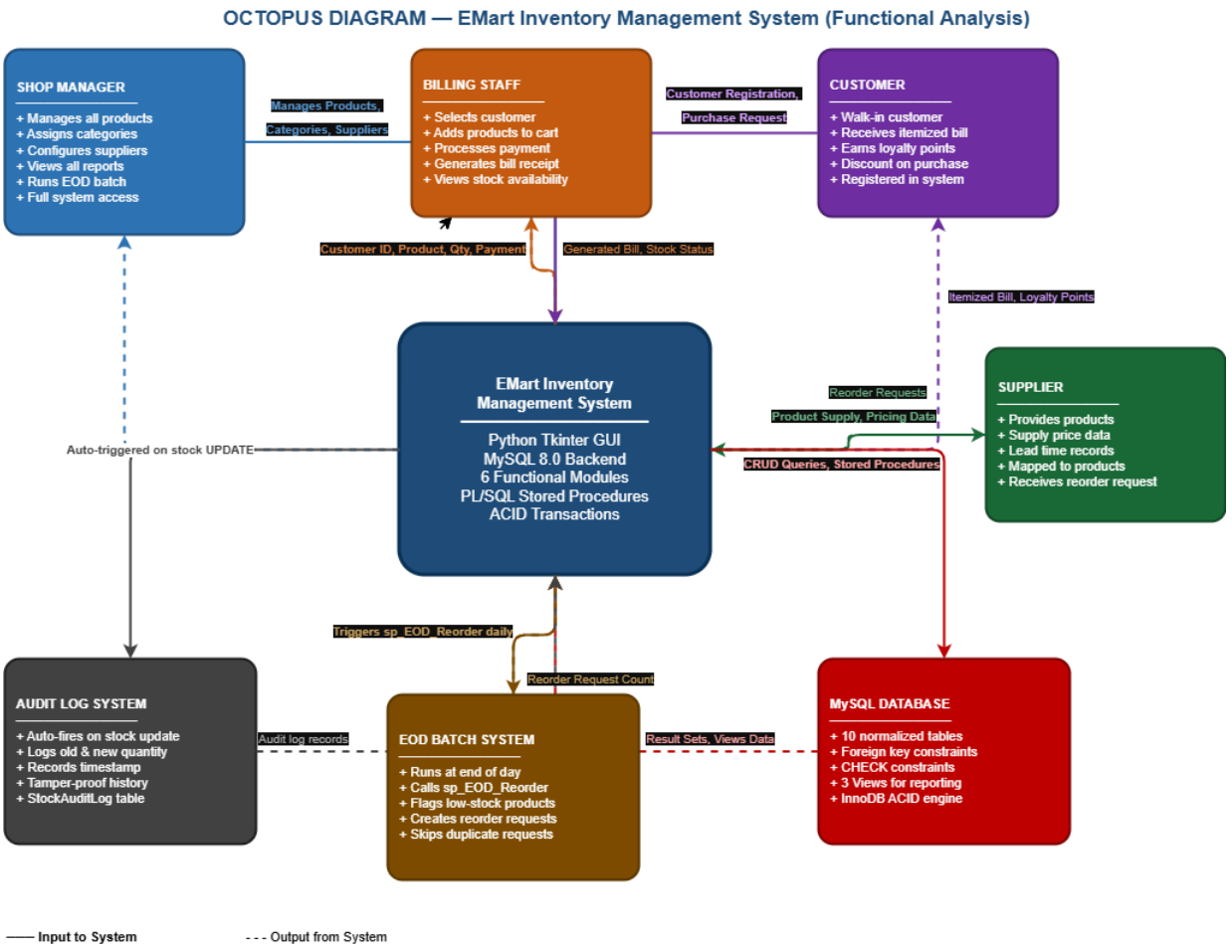
3.2 Octopus Diagram

The Octopus Diagram (Functional Analysis diagram) identifies all external actors that interact with the EMart Inventory Management System. The system is placed at the center, and each 'tentacle' represents a stakeholder or actor with their corresponding interactions.

Central System: EMart Inventory Management System

External Actors (Tentacles):

- Shop Manager: Manages products, categories, suppliers, and views all reports. Has full system access.
- Billing Staff: Creates bills for walk-in customers, processes payments, deducts stock.
- Supplier: Provides products and supply pricing information (data entered by manager).
- Customer: Purchases products, accumulates loyalty points, receives bills.
- MySQL Database: Stores all persistent data, enforces constraints, triggers, and procedures.
- EOD Batch System: Automatically triggered stored procedure that processes reorder requests at day end.
- Audit Log System: Receives trigger-generated entries whenever stock quantities are modified.



CHAPTER 4: SYSTEM DESIGN

4.1 Entity-Relationship (ER) Diagram

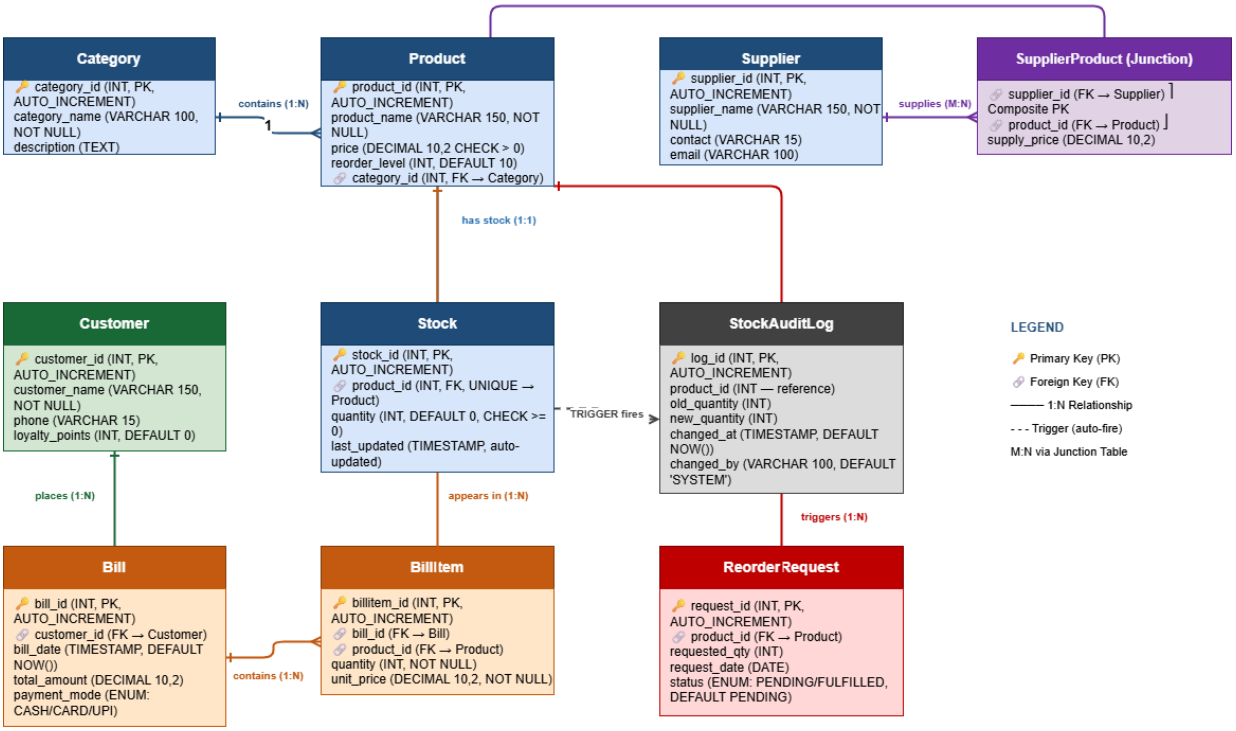
The Entity-Relationship (ER) diagram represents all entities in the EMart Inventory System, their attributes, and the relationships between them. The diagram is aligned with the business and functional goals of managing grocery inventory, billing, and supplier tracking.

Entities and Attributes:

ENTITY	ATTRIBUTES
Category	category_id [PK], category_name, description
Product	product_id [PK], product_name, price, reorder_level, category_id [FK]
Supplier	supplier_id [PK], supplier_name, contact, email
SupplierProduct	supplier_id [FK], product_id [FK], supply_price — Junction Table
Stock	stock_id [PK], product_id [FK, UNIQUE], quantity, last_updated
Customer	customer_id [PK], customer_name, phone, loyalty_points
Bill	bill_id [PK], customer_id [FK], bill_date, total_amount, payment_mode
BillItem	billitem_id [PK], bill_id [FK], product_id [FK], quantity, unit_price
StockAuditLog	log_id [PK], product_id, old_quantity, new_quantity, changed_at, changed_by
ReorderRequest	request_id [PK], product_id [FK], requested_qty, request_date, status

Relationships:

ENTITIES	CARDINALITY	DESCRIPTION
Category → Product	One-to-Many	One category has many products
Supplier ↔ Product	Many-to-Many	Via SupplierProduct junction table
Product → Stock	One-to-One	Each product has exactly one stock record
Customer → Bill	One-to-Many	One customer can have many bills
Bill → BillItem	One-to-Many	One bill contains many line items
Product → BillItem	One-to-Many	One product can appear in many bill items
Product → ReorderRequest	One-to-Many	A product can have multiple reorder requests

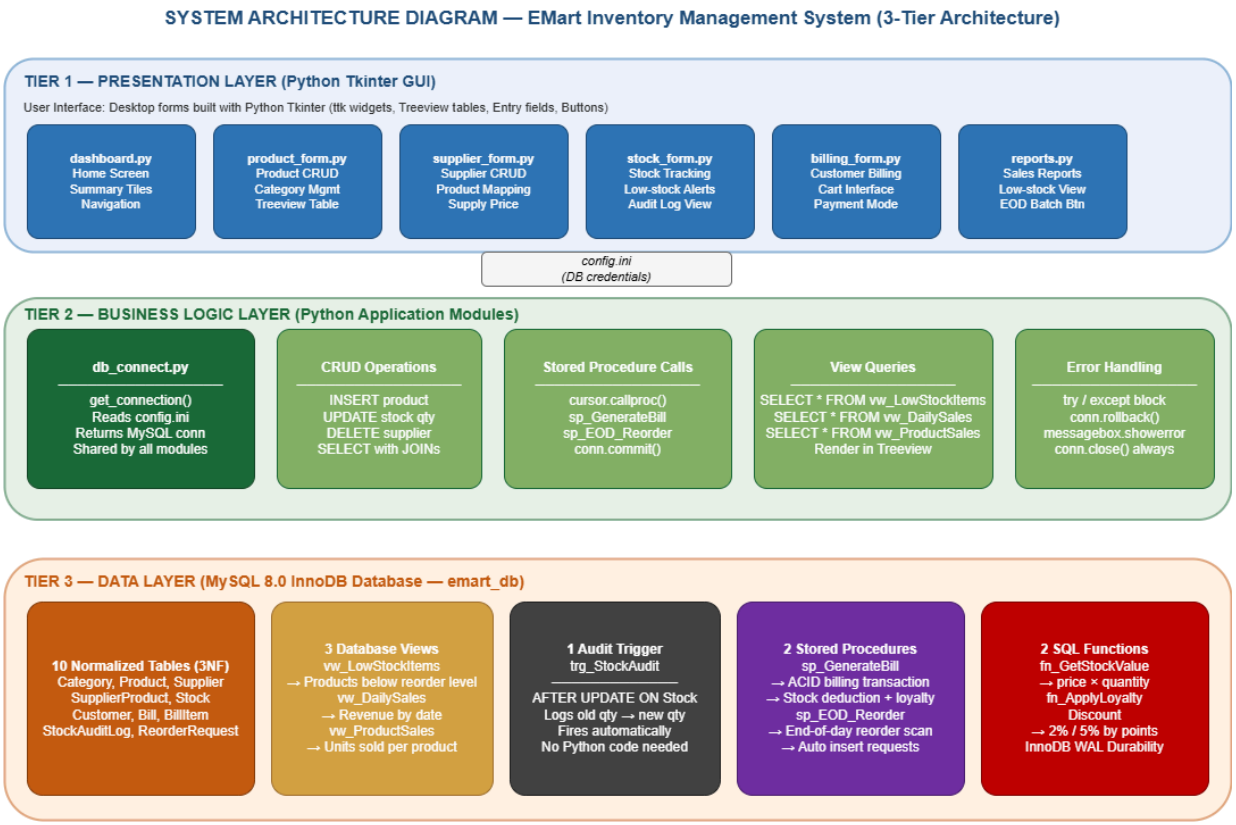


4.2 System Architecture Diagram

The system follows a three-tier architecture consisting of the Presentation Layer (Python Tkinter GUI), the Business Logic Layer (Python modules handling application logic), and the Data Layer (MySQL Database with stored procedures, triggers, views, and functions).

Architecture Layers:

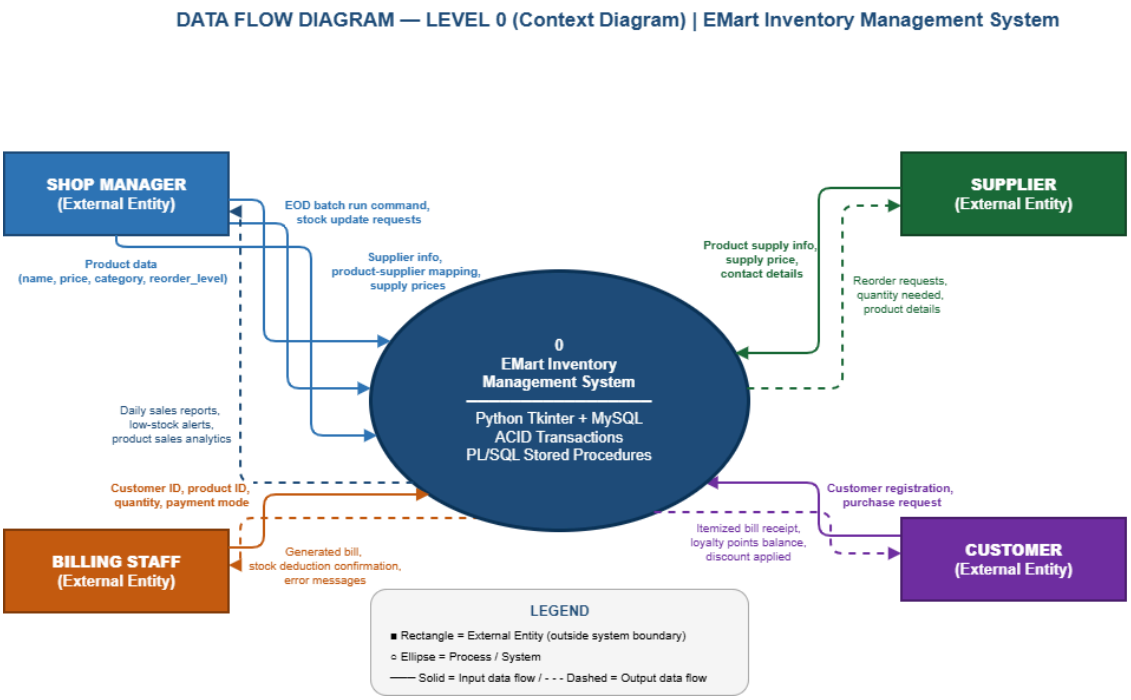
- Presentation Layer (Tier 1): Python Tkinter forms — dashboard.py, product_form.py, supplier_form.py, stock_form.py, billing_form.py, reports.py
- Business Logic Layer (Tier 2): db_connect.py handles MySQL connections; individual form modules implement CRUD logic and call stored procedures.
- Data Layer (Tier 3): MySQL 8.0 database with 10 normalized tables, 3 views, 1 audit trigger, 2 SQL functions, and 2 stored procedures.



4.3 Data Flow Diagram

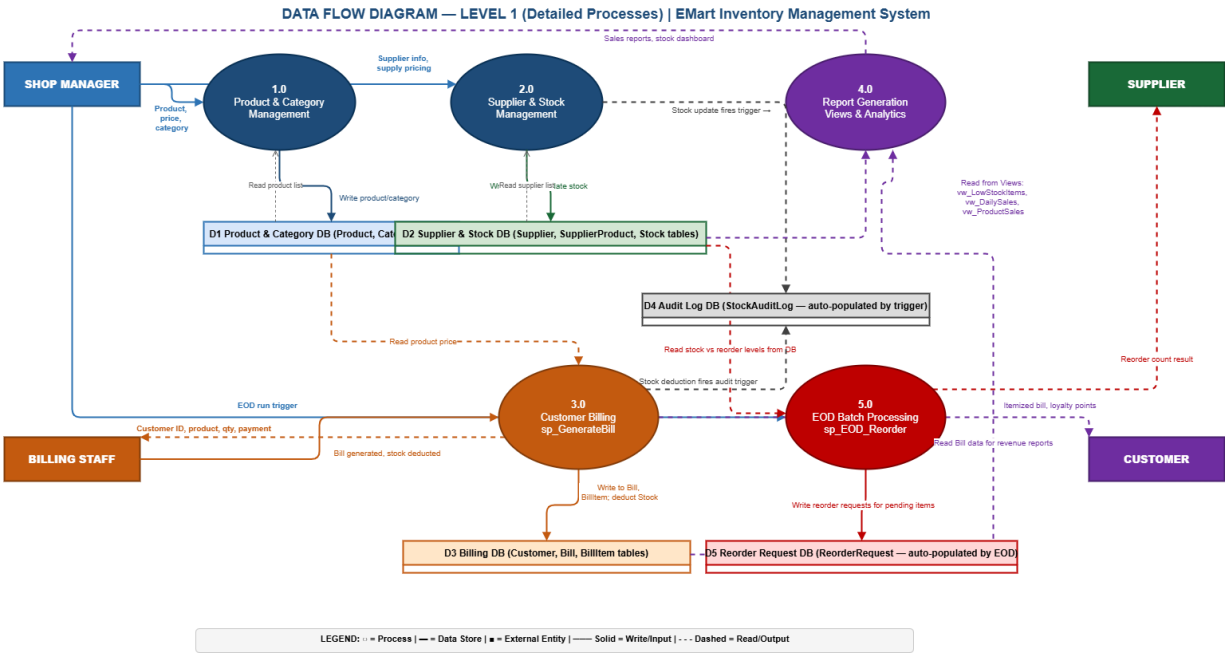
DFD Level 0 (Context Diagram):

The Context Diagram shows the EMart Inventory System as a single process interacting with three external entities: the Shop Manager (who inputs product, supplier, and stock data and receives reports), the Billing Staff (who inputs customer and purchase data and receives bills), and the Customer (who receives bills and loyalty points).



DFD Level 1 (Detailed Processes):

The Level 1 DFD decomposes the system into five sub-processes: (1) Product & Category Management, (2) Supplier & Stock Management, (3) Customer Billing, (4) Report Generation, and (5) EOD Batch Processing. Each sub-process shows its data inputs, outputs, and the data stores it reads from or writes to.



CHAPTER 5: DATABASE DESIGN & NORMALIZATION

5.1 Normalization (1NF to 3NF)

Normalization is the process of organizing a database to reduce redundancy and improve data integrity. The following example demonstrates normalization of the BillItem-related data from Unnormalized Form (UNF) through Third Normal Form (3NF).

Unnormalized Form (UNF) — Table 5.1

Consider an initial raw table where all purchase information is stored in one flat table with repeating groups. This UNF table violates 1NF because product_names and quantities columns contain multiple values.

bill_id	customer_name	customer_phone	product_names	quantities	prices	total
B001	Ravi Kumar	9876543210	Rice, Sugar	2, 1	60, 45	165
B002	Priya Nair	9988776655	Wheat Flour	3	40	120

Table 5.1: Unnormalized Product-Supplier Data

First Normal Form (1NF) — Table 5.2

Rule: Each column must contain atomic (indivisible) values. No repeating groups. Each row must be unique. Now each row has atomic values. However, customer_name and customer_phone are partially dependent on bill_id only, which violates 2NF.

bill_id	customer_name	customer_phone	product_name	qty	price	total
B001	Ravi Kumar	9876543210	Rice	2	60	120
B001	Ravi Kumar	9876543210	Sugar	1	45	45
B002	Priya Nair	9988776655	Wheat Flour	3	40	120

Table 5.2: After 1NF – Atomic Values

Second Normal Form (2NF) — Table 5.3

Rule: Must be in 1NF and every non-key attribute must be fully functionally dependent on the entire primary key (no partial dependencies). We separate customer data into its own table.

bill_id	customer_id	bill_date	total_amount	payment	
B001	C001	2026-02-23	165.00	CASH	
B002	C002	2026-02-23	120.00	UPI	

Table 5.3: After 2NF – Partial Dependency Removed

Third Normal Form (3NF) — Table 5.4

Rule: Must be in 2NF and there must be no transitive dependencies. We extract Category into its own table. The design is now in 3NF — all non-key attributes depend only on their table's primary key.

product_id	product_name	price	reorder_level	category_id
P001	Basmati Rice (5kg)	280.00	10	C01
P002	Sugar (1kg)	45.00	20	C02

Table 5.4: After 3NF – Transitive Dependency Removed

5.2 Complete Table Schemas

The following SQL scripts create all 10 tables in the emart_db database. Tables are created in dependency order — parent tables before child tables.

```
CREATE DATABASE IF NOT EXISTS emart_db;
USE emart_db;

-- Table 1: Category
CREATE TABLE Category (
    category_id   INT AUTO_INCREMENT PRIMARY KEY,
    category_name VARCHAR(100) NOT NULL,
    description   TEXT
);

-- Table 2: Product (depends on Category)
CREATE TABLE Product (
    product_id   INT AUTO_INCREMENT PRIMARY KEY,
    product_name VARCHAR(150) NOT NULL,
    price        DECIMAL(10,2) NOT NULL CHECK (price > 0),
    reorder_level INT DEFAULT 10,
    category_id  INT,
    FOREIGN KEY (category_id) REFERENCES Category(category_id)
);

-- Table 5: Stock (depends on Product)
CREATE TABLE Stock (
    stock_id     INT AUTO_INCREMENT PRIMARY KEY,
    product_id   INT UNIQUE NOT NULL,
    quantity     INT DEFAULT 0,
    last_updated TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    CONSTRAINT chk_qty CHECK (quantity >= 0),
    FOREIGN KEY (product_id) REFERENCES Product(product_id)
);

-- Table 9: StockAuditLog (populated by trigger)
CREATE TABLE StockAuditLog (
    log_id       INT AUTO_INCREMENT PRIMARY KEY,
    product_id   INT,
    old_quantity INT,
    new_quantity INT,
    changed_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    changed_by   VARCHAR(100) DEFAULT 'SYSTEM'
);
```

CHAPTER 6: VIEWS, TRIGGERS & FUNCTIONS

6.1 Views

Database views are virtual tables based on SQL SELECT queries. They simplify complex queries and provide a consistent interface for reporting without exposing underlying table structures. Three views are implemented for enterprise-grade reporting in EMart.

View 1: vw_LowStockItems — Low Stock Alert

This view joins the Product and Stock tables to display all products whose current quantity has fallen below their defined reorder level. The shortage column computes exactly how many units need to be procured.

```
CREATE VIEW vw_LowStockItems AS
SELECT
    p.product_id,
    p.product_name,
    p.reorder_level,
    s.quantity AS current_stock,
    (p.reorder_level - s.quantity) AS shortage
FROM Product p
JOIN Stock s ON p.product_id = s.product_id
WHERE s.quantity < p.reorder_level;
```

View 2: vw_DailySales — Daily Sales Summary

This view aggregates billing data by date to show the total number of bills generated and the total revenue for each day.

```
CREATE VIEW vw_DailySales AS
SELECT
    DATE(bill_date) AS sale_date,
    COUNT(bill_id)  AS total_bills,
    SUM(total_amount) AS total_revenue
FROM Bill
GROUP BY DATE(bill_date);
```

View 3: vw_ProductSales — Product-wise Sales

This view joins BillItem and Product to compute the total units sold and total revenue generated per product across all billing history.

```
CREATE VIEW vw_ProductSales AS
SELECT
    p.product_name,
    SUM(bi.quantity) AS total_units_sold,
    SUM(bi.quantity * bi.unit_price) AS total_revenue
FROM BillItem bi
JOIN Product p ON bi.product_id = p.product_id
GROUP BY p.product_id, p.product_name
ORDER BY total_revenue DESC;
```

6.2 Triggers (Audit)

A database trigger is a procedural code that automatically executes in response to certain events on a table. The following AFTER UPDATE trigger on the Stock table records every stock change in the audit log — without any code in the Python application.

```
DELIMITER $$
CREATE TRIGGER trg_StockAudit
AFTER UPDATE ON Stock
FOR EACH ROW
BEGIN
    IF OLD.quantity <> NEW.quantity THEN
        INSERT INTO StockAuditLog
            (product_id, old_quantity, new_quantity, changed_at, changed_by)
        VALUES
            (OLD.product_id, OLD.quantity, NEW.quantity, NOW(), 'SYSTEM');
    END IF;
END$$
DELIMITER ;
```

6.3 Functions

SQL functions encapsulate reusable computation logic. Two functions are implemented for EMart.

Function 1: fn_GetStockValue — Compute Stock Value

```
DELIMITER $$
CREATE FUNCTION fn_GetStockValue(pid INT)
RETURNS DECIMAL(10,2) DETERMINISTIC
BEGIN
    DECLARE val DECIMAL(10,2);
    SELECT p.price * s.quantity INTO val
    FROM Product p JOIN Stock s ON p.product_id = s.product_id
    WHERE p.product_id = pid;
    RETURN IFNULL(val, 0.00);
END$$
DELIMITER ;
```

Function 2: fn_ApplyLoyaltyDiscount — Apply Discount

```
DELIMITER $$
CREATE FUNCTION fn_ApplyLoyaltyDiscount(
    bill_total DECIMAL(10,2), loyalty_pts INT)
RETURNS DECIMAL(10,2) DETERMINISTIC
BEGIN
    DECLARE discount DECIMAL(10,2) DEFAULT 0;
    IF loyalty_pts >= 500 THEN SET discount = bill_total * 0.05;
    ELSEIF loyalty_pts >= 200 THEN SET discount = bill_total * 0.02;
    END IF;
    RETURN bill_total - discount;
END$$
DELIMITER ;
-- Usage: SELECT fn_ApplyLoyaltyDiscount(1000.00, 600); --> 950.00
```

CHAPTER 7: STORED PROCEDURES & EOD BATCH PROCESSING

Stored Procedures are compiled, reusable SQL programs stored in the database. They encapsulate complex business logic, reduce network traffic by executing logic server-side, and enforce consistent business rules.

7.1 sp_GenerateBill — Billing Transaction Procedure

This is the core billing stored procedure. It wraps the entire billing workflow — creating the bill header, adding bill items, deducting stock — inside a single ACID-compliant database transaction. If any step fails, the entire transaction rolls back, ensuring data consistency.

```
DELIMITER $$
CREATE PROCEDURE sp_GenerateBill(
    IN  p_customer_id  INT,
    IN  p_payment_mode VARCHAR(10),
    IN  p_product_id   INT,
    IN  p_qty          INT,
    OUT p_bill_id      INT,
    OUT p_message      VARCHAR(200)
)
sp_label: BEGIN
    DECLARE v_price   DECIMAL(10,2);
    DECLARE v_total   DECIMAL(10,2);
    DECLARE v_stock   INT;
    DECLARE v_bill_id INT;
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SET p_bill_id = -1;
        SET p_message = 'Transaction failed. All changes rolled back.';
    END;
    -- Check stock availability BEFORE starting transaction
    SELECT quantity INTO v_stock FROM Stock WHERE product_id = p_product_id;
    IF v_stock < p_qty THEN
        SET p_bill_id = -1;
        SET p_message = CONCAT('Insufficient stock. Available: ', v_stock);
        LEAVE sp_label;
    END IF;
    START TRANSACTION;
    SELECT price INTO v_price FROM Product WHERE product_id = p_product_id;
    SET v_total = v_price * p_qty;
    INSERT INTO Bill (customer_id, total_amount, payment_mode)
        VALUES (p_customer_id, v_total, p_payment_mode);
    SET v_bill_id = LAST_INSERT_ID();
    INSERT INTO BillItem (bill_id, product_id, quantity, unit_price)
        VALUES (v_bill_id, p_product_id, p_qty, v_price);
    UPDATE Stock SET quantity = quantity - p_qty WHERE product_id = p_product_id;
    UPDATE Customer SET loyalty_points = loyalty_points + FLOOR(v_total/100)
        WHERE customer_id = p_customer_id;
    COMMIT;
    SET p_bill_id = v_bill_id;
    SET p_message = CONCAT('Bill #', v_bill_id, ' generated successfully.');
END sp_label$$
DELIMITER ;
```

7.2 sp_EOD_Reorder — End-of-Day Batch Processing

When called at the end of each business day, this procedure scans all products, identifies those with stock below their reorder level, and automatically inserts reorder requests into the ReorderRequest table. It skips products that already have a pending request for today to avoid duplicates.

```
DELIMITER $$
CREATE PROCEDURE sp_EOD_Reorder()
BEGIN
    DECLARE v_count INT DEFAULT 0;
    INSERT INTO ReorderRequest (product_id, requested_qty, request_date, status)
    SELECT
        s.product_id,
        (p.reorder_level * 2) - s.quantity AS requested_qty,
        CURDATE(),
        'PENDING'
    FROM Stock s JOIN Product p ON s.product_id = p.product_id
    WHERE s.quantity < p.reorder_level
    AND s.product_id NOT IN (
        SELECT product_id FROM ReorderRequest
        WHERE request_date = CURDATE() AND status = 'PENDING'
    );
    SET v_count = ROW_COUNT();
    SELECT CONCAT(v_count, ' products flagged for reorder today.') AS Result;
END$$
DELIMITER ;
```

CHAPTER 8: ACID PROPERTIES DEMONSTRATION

ACID properties are the foundational guarantees that define reliable database transactions. ACID stands for Atomicity, Consistency, Isolation, and Durability.

8.1 Atomicity

Atomicity ensures that a transaction is treated as a single, indivisible unit of work. Either all operations complete successfully, or none take effect.

```
-- ATOMICITY DEMO
START TRANSACTION;
    INSERT INTO Bill (customer_id, total_amount, payment_mode)
        VALUES (1, 560.00, 'UPI');
    SET @bill_id = LAST_INSERT_ID();
    INSERT INTO BillItem (bill_id, product_id, quantity, unit_price)
        VALUES (@bill_id, 2, 4, 140.00);
    UPDATE Stock SET quantity = quantity - 4 WHERE product_id = 2;
COMMIT; -- All 3 operations succeed together
-- ROLLBACK; -- All 3 operations undo together
```

8.2 Consistency

Consistency ensures that a transaction brings the database from one valid state to another. The CHECK constraint on Stock ensures quantity never goes negative.

```
-- CONSISTENCY DEMO
START TRANSACTION;
UPDATE Stock SET quantity = quantity - 50 WHERE product_id = 1;
-- ERROR: Check constraint chk_qty violated (quantity >= 0)
ROLLBACK;
```

8.3 Isolation

Isolation ensures concurrent transactions execute as if they were the only transaction running. MySQL's default isolation level is REPEATABLE READ.

```
-- SESSION 1: Billing Clerk A
SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ;
START TRANSACTION;
UPDATE Stock SET quantity = quantity - 3 WHERE product_id = 5; -- NOT YET COMMITTED

-- SESSION 2: Billing Clerk B reads stock for same product
SELECT quantity FROM Stock WHERE product_id = 5;
-- Returns ORIGINAL quantity (isolation prevents dirty read)

-- Session 1 commits:
COMMIT; -- Now Session 2 sees updated value on next read
```

8.4 Durability

Durability guarantees that once committed, a transaction remains committed even in the event of system failure. MySQL achieves this through InnoDB's Write-Ahead Log (WAL) mechanism.

```
-- DURABILITY DEMO
START TRANSACTION;
INSERT INTO Bill (customer_id, total_amount, payment_mode)
    VALUES (3, 750.00, 'CARD');
COMMIT; -- After this point, the record is durable
```

```
-- Even if MySQL server restarts, this record persists.
SHOW TABLE STATUS WHERE Name = 'Bill'; -- Engine: InnoDB (WAL-based durability)
```

8.5 ACID Properties Summary

PROPERTY	GUARANTEE	EMART EXAMPLE	SQL MECHANISM
Atomicity	All-or-nothing execution	Billing: Bill + BillItem + Stock deduct all succeed or all rollback	START TRANSACTION / COMMIT / ROLLBACK
Consistency	Valid state before and after	Stock quantity can never go below 0	CHECK constraint on Stock.quantity
Isolation	Concurrent transactions don't interfere	Two billing clerks don't see each other's uncommitted deductions	REPEATABLE READ isolation level
Durability	Committed data persists forever	A committed bill persists through server restart	InnoDB Write-Ahead Log (WAL)

Table 8.1: ACID Properties Summary

CHAPTER 9: MODULE DESCRIPTION

The EMart Inventory Management System is organized into six Python modules, each corresponding to a specific business function. All modules share the db_connect.py database connection utility and interact with the MySQL back-end.

Module 1: db_connect.py — Database Connection Utility

This module provides a centralized function get_connection() that establishes and returns a MySQL database connection. All other modules import this function instead of hard-coding connection parameters. This follows the Single Responsibility Principle and makes configuration changes a one-line update.

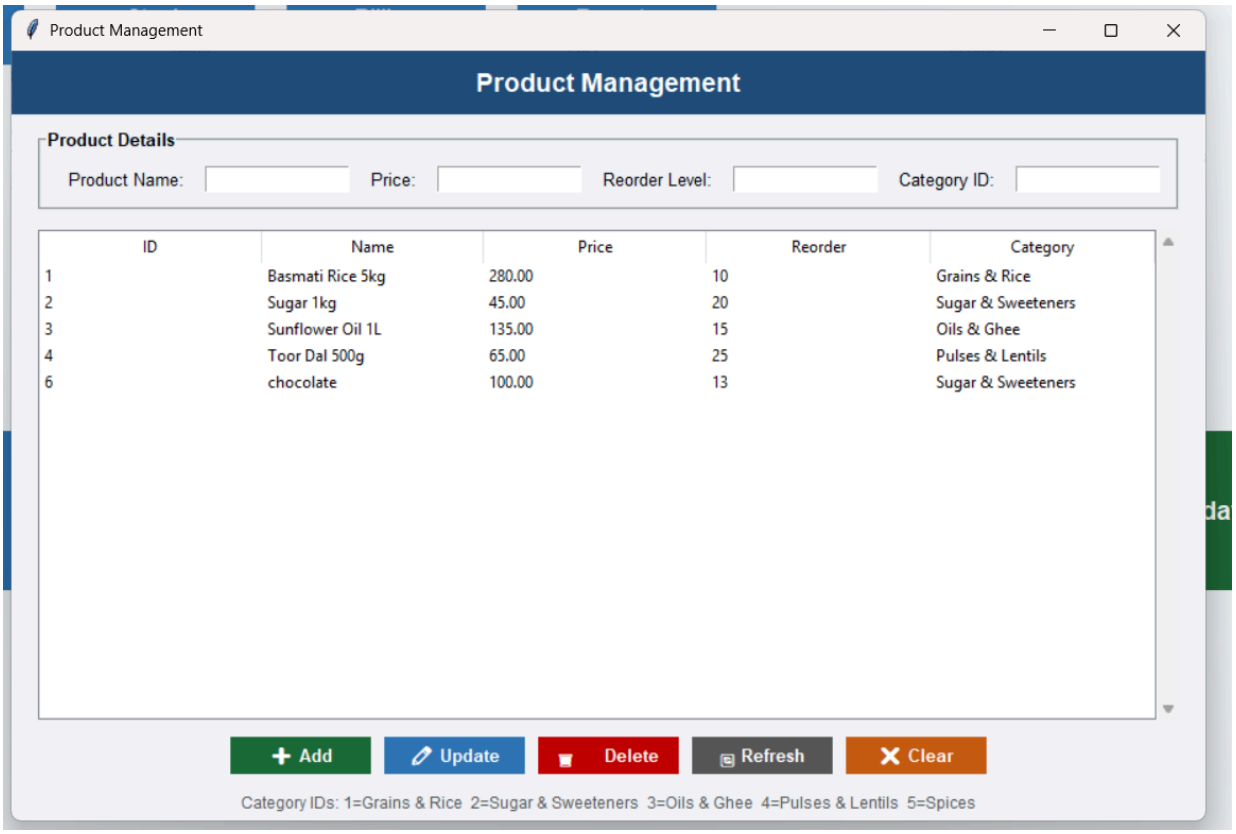
```
import mysql.connector
import configparser, os

def get_connection():
    """Reads DB credentials from config.ini and returns a MySQL connection."""
    config = configparser.ConfigParser()
    config_path = os.path.join(os.path.dirname(__file__), '..', 'config.ini')
    config.read(config_path)
    return mysql.connector.connect(
        host     = config['mysql']['host'],
        user     = config['mysql']['user'],
        password = config['mysql']['password'],
        database = config['mysql']['database']
    )
```

Module 2: dashboard.py — Main Home Screen

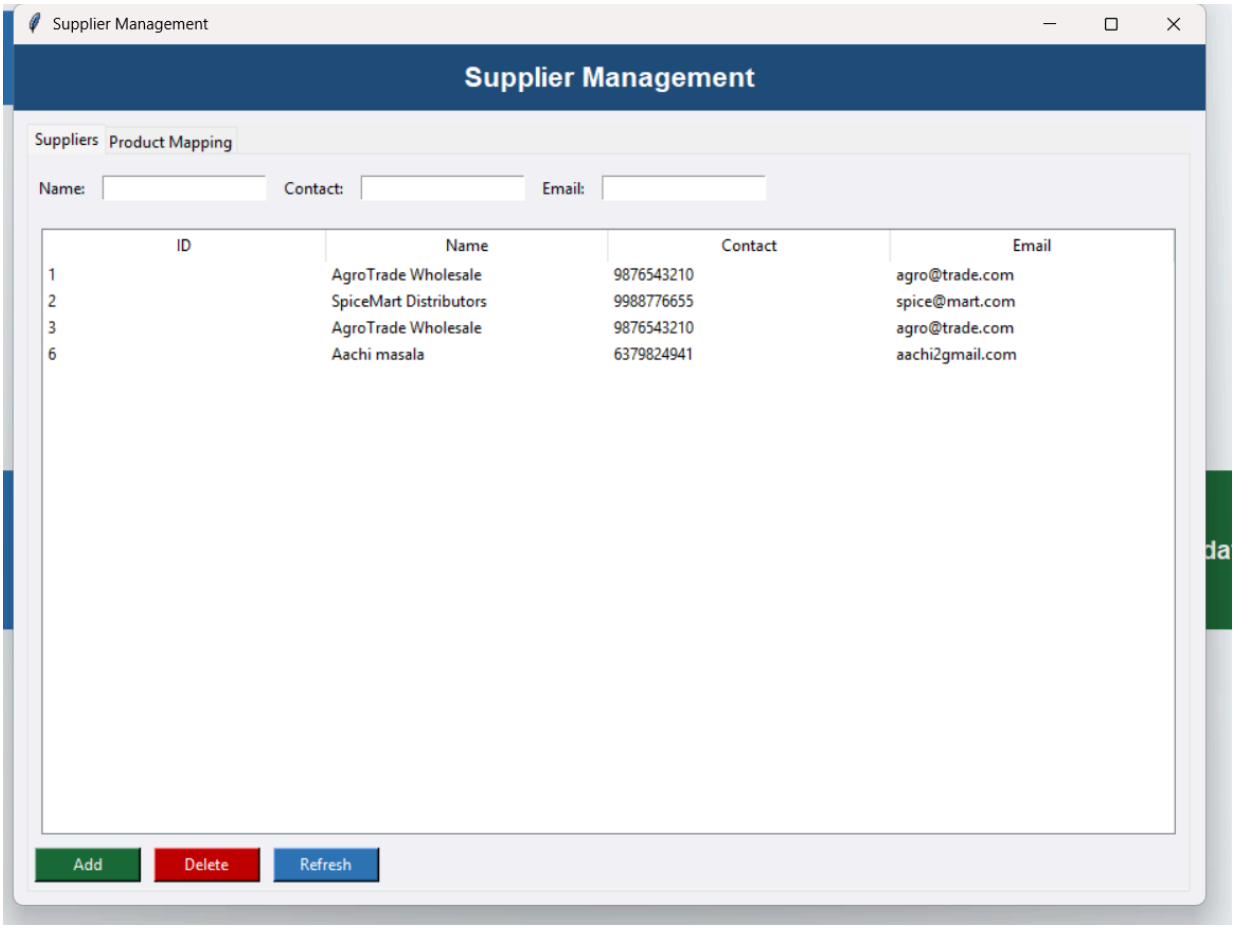
The dashboard is the entry point of the GUI application. It displays a welcome screen with navigation buttons to all other modules. It shows live summary tiles: total products, low-stock

alert count (from vw_LowStockItems), and today's revenue (from vw_DailySales). The layout uses a Tkinter Frame-based structure with a top navigation bar and a central button grid.



Module 3: product_form.py — Product & Category CRUD

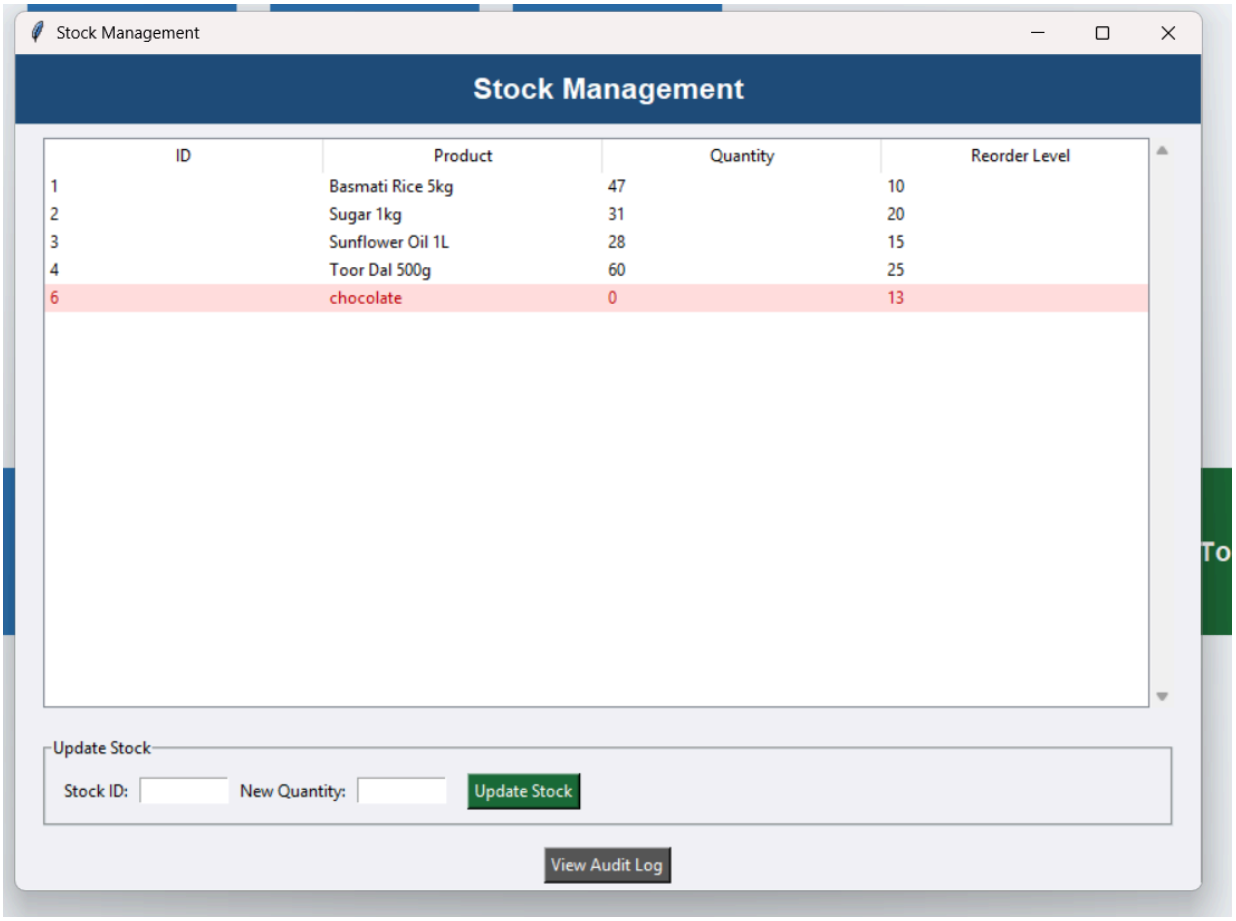
This module provides a full CRUD interface for managing product records. It consists of a Treeview table that displays all products, input fields for product details, and Add/Update/Delete/Search buttons. When a row in the Treeview is selected, the fields auto-populate for editing. Products are joined with their Category name for display.



Module 4: stock_form.py — Stock Management

This module displays current stock levels for all products using a Treeview joined with the vw_LowStockItems view to highlight low-stock items in red. Staff can update quantities, and every update automatically fires the trg_StockAudit trigger to log the change. A separate Audit

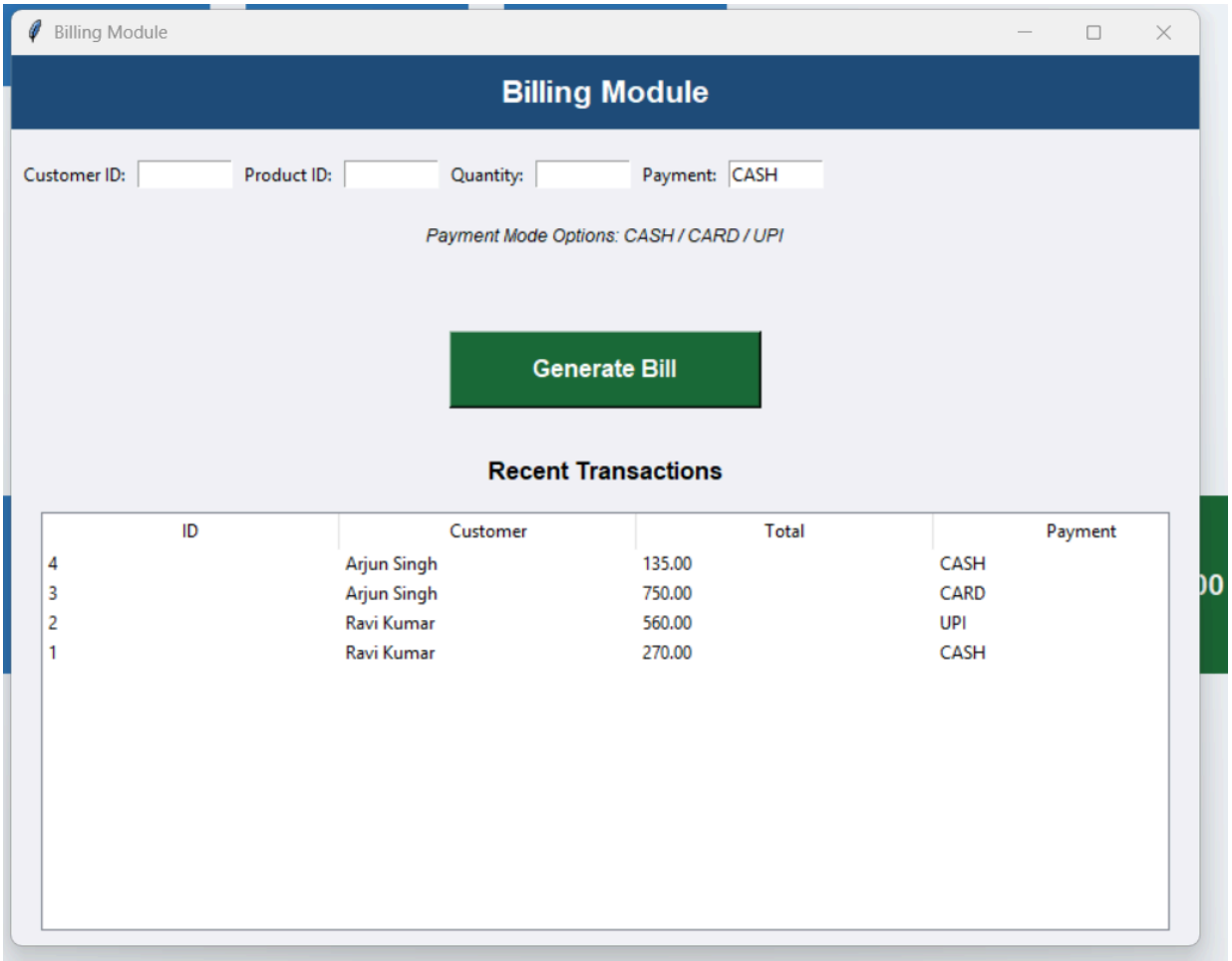
Log button opens a view of the StockAuditLog table.



Module 5: billing_form.py — Customer Billing

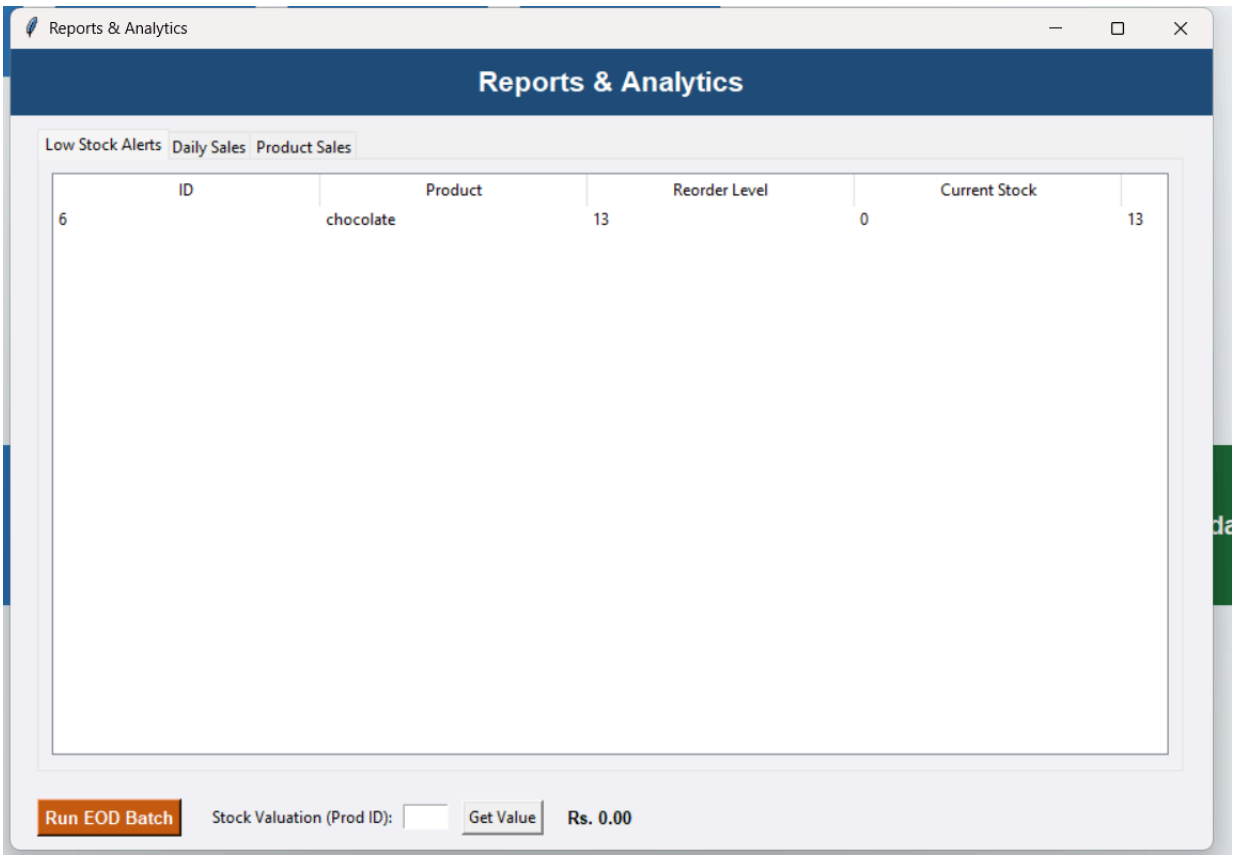
This is the most critical module. It allows billing staff to select a customer, add products to a bill cart with quantity and real-time price lookup, and generate the final bill by calling the sp_GenerateBill stored procedure. The module displays a running total and handles error scenarios (e.g., insufficient stock) with clear user feedback. All business logic is delegated

entirely to the stored procedure.



Module 6: reports.py — Reports & Views Dashboard

This module provides a read-only reporting interface with three tabs: Low Stock Alerts (from vw_LowStockItems), Daily Sales (from vw_DailySales), and Product Sales (from vw_ProductSales). It also provides an EOD Batch button that calls sp_EOD_Reorder() with one click, with a confirmation dialog and result summary.



CHAPTER 10: CONCLUSION

The EMart Grocery Shop Inventory Management System successfully demonstrates the development of a fully functional, enterprise-grade GUI database application using Python Tkinter and MySQL. The project achieves all stated objectives — from designing a normalized 3NF database schema to implementing stored procedures, triggers, views, and ACID-compliant transactions.

The system digitizes the core operations of a retail grocery business: product catalogue management, real-time stock tracking, automated supplier mapping, customer billing with loyalty points, and end-of-day batch reorder processing. The audit trigger ensures a complete, tamper-proof log of all inventory changes, meeting enterprise compliance requirements. The stored procedures encapsulate complex multi-table business logic server-side, improving both performance and data integrity.

From an academic perspective, this project provides hands-on experience with relational database design principles including normalization, entity-relationship modelling, and constraint management. The implementation of all four ACID properties through annotated

transaction scripts provides a deep, practical understanding of transactional database systems.

Future Enhancements

- Barcode Scanner Integration: Integrate USB barcode scanner support in the billing module for faster product selection.
- GST Computation: Add GST tax calculation (5%, 12%, 18% slabs) based on product category to the billing procedure.
- User Authentication: Implement role-based login (Manager, Billing Staff) with hashed password storage.
- Email / SMS Notifications: Automatically email daily sales reports to the manager and send low-stock SMS alerts.
- Supplier Auto-Email: Automatically send reorder requests via email to suppliers when the EOD batch procedure runs.
- Web Interface: Migrate the Tkinter GUI to a Flask or Django web application for multi-device access.
- Business Analytics Dashboard: Add visual charts (bar charts, line graphs) for revenue trends using Matplotlib or Chart.js.

Limitations

- The current system is a single-user desktop application and does not support concurrent multi-user access in its Tkinter form.
- The sp_GenerateBill procedure currently handles single-product billing per call; a real-world implementation would process a full cart with multiple items in one atomic transaction.
- There is no built-in backup and restore functionality; database administrators must manage backups manually using mysqldump.

CHAPTER 11: REFERENCES

[1] Ramakrishnan, R., & Gehrke, J. (2002). Database Management Systems (3rd ed.). McGraw-Hill Education.

[2] Elmasri, R., & Navathe, S. B. (2015). Fundamentals of Database Systems (7th ed.). Pearson Education.

[3] Oracle Corporation. (2023). MySQL 8.0 Reference Manual. Retrieved from https://dev.mysql.com/doc/refman/8.0/en/

[4] Python Software Foundation. (2024). tkinter — Python interface to Tcl/Tk. Retrieved from https://docs.python.org/3/library/tkinter.html

[5] MySQL Connector/Python Developer Guide. (2023). Oracle Corporation. Retrieved from https://dev.mysql.com/doc/connector-python/en/

[6] Date, C. J. (2003). An Introduction to Database Systems (8th ed.). Addison-Wesley.

[7] Schwartz, B., Zaitsev, P., & Tkachenko, V. (2012). High Performance MySQL (3rd ed.). O'Reilly Media.

[8] Garcia-Molina, H., Ullman, J. D., & Widom, J. (2008). Database Systems: The Complete Book (2nd ed.). Prentice Hall.

CHAPTER 12: APPENDICES

12.1 Code — main.py (Application Entry Point)

```
"""
EMart Inventory Management System
Entry Point — Run this file to launch the application.
Requirements: pip install mysql-connector-python
"""
import sys, os
sys.path.insert(0, os.path.dirname(__file__))
from app.dashboard import Dashboard

def main():
    app = Dashboard()
    app.mainloop()

if __name__ == "__main__":
    main()
```

12.1.1 config.ini — Database Configuration

```
# config.ini — NEVER hard-code passwords in source code
# Add config.ini to .gitignore to prevent accidental upload
[mysql]
host     = localhost
user     = root
password = your_mysql_root_password
database = emart_db
```

12.1.2 Sample Data — INSERT Scripts

```
-- Sample Categories
INSERT INTO Category (category_name, description) VALUES
('Grains & Rice', 'All types of rice and cereals'),
('Sugar & Sweeteners','Sugar, jaggery, honey'),
('Oils & Ghee', 'Cooking oils, ghee, vanaspati'),
('Pulses & Lentils', 'Dal, lentils, beans'),
('Spices', 'All masalas and spices');

-- Sample Products
INSERT INTO Product (product_name, price, reorder_level, category_id) VALUES
('Basmati Rice 5kg', 280.00, 10, 1),
('Sugar 1kg',         45.00, 20, 2),
('Sunflower Oil 1L', 135.00, 15, 3),
('Toor Dal 500g',      65.00, 25, 4),
('Turmeric Powder 100g', 28.00, 30, 5);

-- Initial Stock
INSERT INTO Stock (product_id, quantity) VALUES
(1, 50), (2, 8), (3, 30), (4, 60), (5, 5);

-- Sample Customers
INSERT INTO Customer (customer_name, phone, loyalty_points) VALUES
('Ravi Kumar',  '9876500001', 150),
('Priya Nair',  '9876500002', 520),
('Arjun Singh', '9876500003',  80);
```

12.2 Screenshots of database

```
mysql> show tables;
+--------------------+
| Tables_in_emart_db |
+--------------------+
| bill               |
| billitem           |
| category           |
| customer           |
| product            |
| reorderrequest     |
| stock              |
| stockauditlog      |
| supplier           |
| supplierproduct    |
| vw_dailysales      |
| vw_lowstockitems   |
| vw_productsales    |
+--------------------+
13 rows in set (0.01 sec)
```

```
mysql> SELECT * FROM Category;
+-------------+--------------------+-------------------------------+
| category_id | category_name      | description                   |
+-------------+--------------------+-------------------------------+
|           1 | Grains & Rice      | All types of rice and cereals |
|           2 | Sugar & Sweeteners | Sugar, jaggery, honey         |
|           3 | Oils & Ghee        | Cooking oils, ghee, vanaspati |
|           4 | Pulses & Lentils   | Dal, lentils, beans           |
|           5 | Spices             | All masalas and spices        |
+-------------+--------------------+-------------------------------+
5 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM Product;
+------------+------------------+--------+---------------+-------------+
| product_id | product_name     | price  | reorder_level | category_id |
+------------+------------------+--------+---------------+-------------+
|          1 | Basmati Rice 5kg | 280.00 |            10 |           1 |
|          2 | Sugar 1kg        |  45.00 |            20 |           2 |
|          3 | Sunflower Oil 1L | 135.00 |            15 |           3 |
|          4 | Toor Dal 500g    |  65.00 |            25 |           4 |
|          6 | chocolate        | 100.00 |            13 |           2 |
+------------+------------------+--------+---------------+-------------+
5 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM Supplier;
+-------------+-----------------------+------------+-----------------+
| supplier_id | supplier_name         | contact    | email           |
+-------------+-----------------------+------------+-----------------+
|           1 | AgroTrade Wholesale   | 9876543210 | agro@trade.com  |
|           2 | SpiceMart Distributors | 9988776655 | spice@mart.com  |
|           3 | AgroTrade Wholesale   | 9876543210 | agro@trade.com  |
|           6 | Aachi masala          | 6379824941 | aachi2gmail.com |
+-------------+-----------------------+------------+-----------------+
4 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM SupplierProduct;
+-------------+------------+---------------+
| supplier_id | product_id | supply_price  |
+-------------+------------+---------------+
|           1 |          1 |        240.00 |
|           1 |          2 |         38.00 |
|           1 |          3 |        115.00 |
|           1 |          4 |         55.00 |
+-------------+------------+---------------+
4 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM Stock;
+----------+------------+----------+---------------------+
| stock_id | product_id | quantity | last_updated        |
+----------+------------+----------+---------------------+
|        1 |          1 |       47 | 2026-03-06 23:59:32 |
|        2 |          2 |       31 | 2026-03-07 10:16:31 |
|        3 |          3 |       28 | 2026-03-07 00:01:03 |
|        4 |          4 |       60 | 2026-03-06 23:57:43 |
|        6 |          6 |        0 | 2026-03-07 10:25:30 |
+----------+------------+----------+---------------------+
5 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM Customer;
+-------------+---------------+------------+----------------+
| customer_id | customer_name | phone      | loyalty_points |
+-------------+---------------+------------+----------------+
|           1 | Ravi Kumar    | 9876500001 |            152 |
|           2 | Priya Nair    | 9876500002 |            520 |
|           3 | Arjun Singh   | 9876500003 |             81 |
+-------------+---------------+------------+----------------+
3 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM Bill;
+---------+-------------+---------------------+--------------+--------------+
| bill_id | customer_id | bill_date           | total_amount | payment_mode |
+---------+-------------+---------------------+--------------+--------------+
|       1 |           1 | 2026-03-07 00:01:03 |       270.00 | CASH         |
|       2 |           1 | 2026-03-07 00:01:18 |       560.00 | UPI          |
|       3 |           3 | 2026-03-07 00:01:33 |       750.00 | CARD         |
|       4 |           3 | 2026-03-07 10:16:31 |       135.00 | CASH         |
+---------+-------------+---------------------+--------------+--------------+
4 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM BillItem;
+-------------+---------+------------+----------+------------+
| billitem_id | bill_id | product_id | quantity | unit_price |
+-------------+---------+------------+----------+------------+
|           1 |       1 |          3 |        2 |     135.00 |
|           2 |       2 |          2 |        4 |     140.00 |
|           3 |       4 |          2 |        3 |      45.00 |
+-------------+---------+------------+----------+------------+
3 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM StockAuditLog;
+--------+------------+--------------+--------------+---------------------+------------+
| log_id | product_id | old_quantity | new_quantity | changed_at          | changed_by |
+--------+------------+--------------+--------------+---------------------+------------+
|      1 |          1 |           50 |           47 | 2026-03-06 23:59:32 | SYSTEM     |
|      2 |          3 |           30 |           28 | 2026-03-07 00:01:03 | SYSTEM     |
|      3 |          2 |            8 |            4 | 2026-03-07 00:01:18 | SYSTEM     |
|      4 |          2 |            4 |           34 | 2026-03-07 10:14:54 | SYSTEM     |
|      5 |          2 |           34 |           31 | 2026-03-07 10:16:31 | SYSTEM     |
+--------+------------+--------------+--------------+---------------------+------------+
5 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM ReorderRequest;
+------------+------------+---------------+--------------+---------+
| request_id | product_id | requested_qty | request_date | status  |
+------------+------------+---------------+--------------+---------+
|          1 |          2 |            32 | 2026-03-07   | PENDING |
+------------+------------+---------------+--------------+---------+
1 row in set (0.00 sec)
```

```
mysql> SELECT * FROM vw_LowStockItems;
+------------+--------------+---------------+---------------+----------+
| product_id | product_name | reorder_level | current_stock | shortage |
+------------+--------------+---------------+---------------+----------+
|          6 | chocolate    |            13 |             0 |       13 |
+------------+--------------+---------------+---------------+----------+
1 row in set (0.00 sec)
```

```
mysql> SELECT * FROM vw_DailySales;
+------------+--------------+----------------+
| sale_date  | total_bills  | total_revenue  |
+------------+--------------+----------------+
| 2026-03-07 |            4 |        1715.00 |
+------------+--------------+----------------+
1 row in set (0.00 sec)
```

```
mysql> SELECT * FROM vw_ProductSales;
+------------------+------------------+---------------+
| product_name     | total_units_sold | total_revenue |
+------------------+------------------+---------------+
| Sugar 1kg        |                7 |        695.00 |
| Sunflower Oil 1L |                2 |        270.00 |
+------------------+------------------+---------------+
2 rows in set (0.00 sec)
```